279. Perfect Squares - Explanation

Description

You are given an integer n, return the least number of perfect square numbers that sum to n.

A perfect square is an integer that is the square of an integer. For example, 1, 4, 9, 16, 25... are perfect squares.

Example 1:
Input: n = 13
Output: 2
Explanation: 13 = 4 + 9.

Example 2:
Input: n = 6
Output: 3
Explanation: 6 = 4 + 1 + 1.

Constraints:
1 <= n <= 10,000

1. Recursion
Intuition
We want to express n as a sum of the fewest perfect squares. At each step, we can subtract any perfect square that fits, then recursively solve for the remainder. By trying all possible perfect squares and taking the minimum, we find the optimal answer. This brute-force approach explores all combinations but results in repeated subproblems.

Algorithm
Define a recursive function that takes a target value.
Base case: if target is 0, return 0 (no squares needed).
Initialize the result to target (the worst case is using all 1s).
For each perfect square i*i that does not exceed target, recursively solve for (target - i*i) and update the result with 1 + the recursive result.
Return the minimum count found.

```
public class Solution {
    public int numSquares(int n) {
        return dfs(n);
    }

    private int dfs(int target) {
        if (target == 0) {
            return 0;
        }

        int res = target;
        for (int i = 1; i * i <= target; i++) {
            res = Math.min(res, 1 + dfs(target - i * i));
        }
        return res;
```

```
    }
}
```
Time & Space Complexity
Time complexity: O(n^Math.sqrt(n))
Space complexity: O(n) for recursion stack.

2. Dynamic Programming (Top-Down)
Intuition
The recursive solution recomputes the same subproblems many times. By caching results in a memoization table, we avoid redundant work. Each unique target value is solved once, and subsequent calls return the cached result. This transforms the exponential time complexity into polynomial.

Algorithm
Create a memoization dictionary to store results for each target value.
Define a recursive function. If target is 0, return 0. If target is in memo, return the cached value.
Initialize result to target.
For each perfect square i*i up to target, compute 1 + dfs(target - i*i) and track the minimum.
Store the result in memo and return it.
Call the recursive function with n.

```
public class Solution {
    Map<Integer, Integer> memo = new HashMap<>();

    private int dfs(int target) {
        if (target == 0) return 0;
        if (memo.containsKey(target)) return memo.get(target);

        int res = target;
        for (int i = 1; i * i <= target; i++) {
            res = Math.min(res, 1 + dfs(target - i * i));
        }

        memo.put(target, res);
        return res;
    }

    public int numSquares(int n) {
        return dfs(n);
    }
}
```
Time & Space Complexity
Time complexity: O(n * Math.sqrt(n))
Space complexity: O(n)

3. Dynamic Programming (Bottom-Up)
Intuition
Instead of solving top-down with recursion, we can build the solution bottom-up. We compute the minimum number of squares for every value from 1 to n, using previously computed results. For each target, we try subtracting every perfect square and take the minimum result plus one.

Algorithm
Create a dp array of size n+1, initialized to n (worst case of all 1s). Set dp[0] = 0.
For each target from 1 to n, iterate through all perfect squares s*s that do not exceed target.
Update dp[target] = min(dp[target], 1 + dp[target - s*s]).
Return dp[n].

```
public class Solution {
    public int numSquares(int n) {
        int[] dp = new int[n + 1];
        Arrays.fill(dp, n);
        dp[0] = 0;

        for (int target = 1; target <= n; target++) {
            for (int s = 1; s * s <= target; s++) {
                dp[target] = Math.min(dp[target], 1 + dp[target - s * s]);
            }
        }

        return dp[n];
    }
}
```

Time & Space Complexity
Time complexity: O(n * Math.sqrt(n))
Space complexity: O(n)

4. Breadth First Search
Intuition
We can view this as a shortest path problem. Starting from 0, each step adds a perfect square. BFS explores all sums reachable with 1 square, then 2 squares, and so on. The first time we reach n, we have found the minimum number of squares. Using a set to track visited values prevents processing the same sum multiple times.

Algorithm
Initialize a queue with 0 and a set to track seen values.
Process the queue level by level, incrementing the count at each level.
For each current sum, try adding every perfect square s*s such that current + s*s <= n.
If current + s*s equals n, return the current count.
If the new sum has not been seen, add it to the set and enqueue it.
Continue until n is reached.

```
public class Solution {
    public int numSquares(int n) {
        Queue<Integer> q = new LinkedList<>();
        Set<Integer> seen = new HashSet<>();

        int res = 0;
        q.offer(0);
        while (!q.isEmpty()) {
            res++;
```

```
            for (int i = q.size(); i > 0; i--) {
                int cur = q.poll();
                for (int s = 1; s * s + cur <= n; s++) {
                    int next = cur + s * s;
                    if (next == n) return res;
                    if (!seen.contains(next)) {
                        q.offer(next);
                        seen.add(next);
                    }
                }
            }
        }
        return res;
    }
}
```

Time & Space Complexity

Time complexity:

$O(n * Math.sqrt(n))$

Space complexity: $O(n)$

5. Math

Intuition

Lagrange's four square theorem states that every positive integer can be expressed as the sum of at most four perfect squares. Using additional number theory, we can determine the exact answer in constant time. If n is a perfect square, the answer is 1. If n can be written as the sum of two squares, the answer is 2. If n is of the form 4^k(8m+7), the answer is 4. Otherwise, the answer is 3.

Algorithm

Remove all factors of 4 from n (divide by 4 while divisible).
If the reduced n is congruent to 7 mod 8, return 4.
Check if the original n is a perfect square. If so, return 1.
Check if n can be expressed as the sum of two squares by testing all possible first squares. If so, return 2.
Otherwise, return 3.

```
public class Solution {
    public int numSquares(int n) {
        while (n % 4 == 0) {
            n /= 4;
        }

        if (n % 8 == 7) {
            return 4;
        }

        if (isSquareNum(n)) {
            return 1;
        }

        for (int i = 1; i * i <= n; i++) {
```

```
            if (isSquareNum(n - i * i)) {
                return 2;
            }
        }

        return 3;
    }

    private boolean isSquareNum(int num) {
        int s = (int) Math.sqrt(num);
        return s * s == num;
    }
}
```

Time & Space Complexity

Time complexity: O(Math.sqrt(n))

Space complexity: O(1)

Common Pitfalls

Not Recognizing Overlapping Subproblems

A common mistake is implementing plain recursion without memoization. The recursive solution recomputes the same subproblems many times (e.g., dfs(5) might be called from multiple paths). Without caching results, the solution becomes exponentially slow and will time out on larger inputs.

Incorrect Base Case or Initialization

Forgetting to handle the base case dp[0] = 0 or initializing the DP array incorrectly leads to wrong answers. The value dp[0] = 0 is crucial because it represents that zero squares are needed to sum to zero. Similarly, initializing other values to n (worst case of all 1s) ensures the minimum is correctly computed.

Iterating Over Non-Perfect-Squares

Some implementations mistakenly iterate through all numbers from 1 to target instead of only perfect squares. This wastes computation and can lead to incorrect state transitions. Always ensure the inner loop only considers values i where i * i <= target.